

Автор: Тарин Пламб

Критическая ошибка Ruby on Rails ставит под сомнение каждую загружаемую картинку

Новости

4 августа 2026 года • 7 минут

Уязвимость связана с Active Storage, что позволяет злоумышленникам без аутентификации читать конфиденциальные файлы и потенциально получать доступ к приложениям.

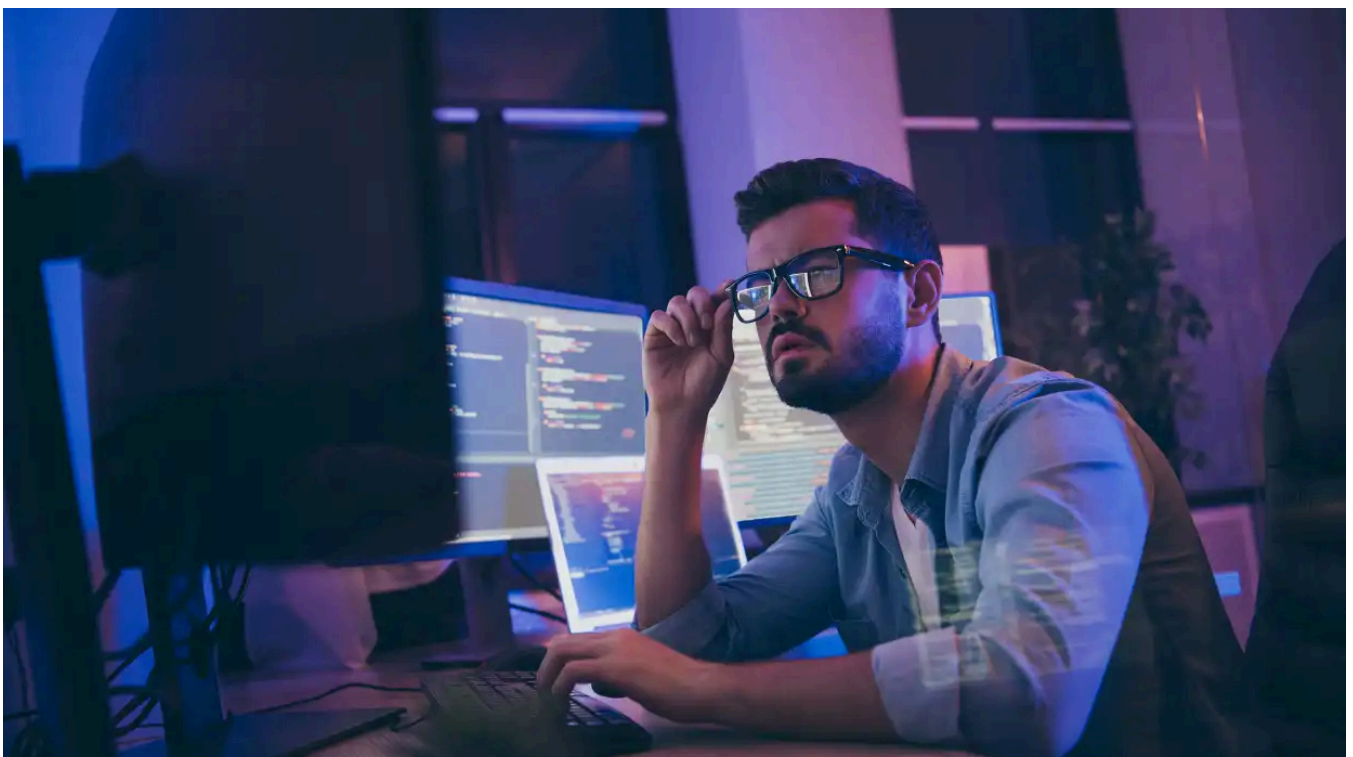


Фото: Роман Самборский / Shutterstock

Новая критическая уязвимость в фреймворке веб-приложений Ruby on Rails («Rails»), **CVE-2026-66066** [<https://www.cve.org/CVERecord?id=CVE-2026-66066>], может превратить, казалось бы, безобидный образ в лазейку для злоумышленников.

Уязвимость CVE высокой степени опасности (9,5 баллов из 10), обнаруженная 30 июля, представляет значительный риск для компаний, использующих приложения, которые обрабатывают загружаемые пользователями изображения в Rails.

Атака, получившая название KindaRails2Shell, нацелена на чрезмерно уязвимый компонент Active Storage фреймворка с открытым исходным кодом.

Злоумышленники, не прошедшие аутентификацию, могут читать конфиденциальные файлы или переходить к удаленному выполнению кода (RCE).

Проблема устранена в версиях 7.2.3.2, 8.0.5.1 и 8.1.3.1 Active Storage. Предприятиям, использующим Rails, следует немедленно обновить программное обеспечение.

«Поцелуй шеф-повара» — это возможность для злоумышленника загрузить изображение, которое на самом деле не является изображением [а представляет собой] код, позволяющий ему украсть секретные данные», — сказал Дэвид Шипли [<https://www.linkedin.com/in/dbshiple/>] из Beauceron Security.

Злоумышленники получают ключ от замка

Ruby on Rails — это серверный фреймворк с открытым исходным кодом, используемый для создания полнофункциональных веб-приложений и интерфейсов прикладного программирования [<https://www.csoonline.com/article/4204548/secure-ai-adoption-starts-with-api-best-practices.html>] (API).

Он популярен среди разработчиков, потому что является масштабируемым, простым в освоении и использовании, поддерживает быструю разработку приложений, опирается на активное сообщество **более 1000 инженеров** [<https://github.blog/engineering/architecture-optimization/building-github-with-ruby-and-rails/>], которые его разрабатывают и поддерживают, и имеет обширную библиотеку из почти двух миллионов строк готового кода.

Уязвимость CVE-2026-66066 затрагивает встроенный в Rails компонент Active Storage, который позволяет пользователям загружать файлы в облачные сервисы или на локальные диски и связывать их со своими приложениями. В частности, уязвимость связана с тем, как Active Storage взаимодействует с libvips библиотекой для обработки изображений [<https://github.com/libvips/libvips>] при создании изображений.

Libvips содержит так называемые «необработанные» операции, которые не защищены от вредоносных входных данных с помощью методов, известных как фаззинг [<https://en.wikipedia.org/wiki/Fuzzing>], которые позволяют выявить случаи, когда программы

аварийно завершают работу, приводят к утечке данных или иным образом ведут себя некорректно. Это делает их небезопасными для использования с ненадежным контентом, но Active Storage не отключает их должным образом.

«CVE-2026-66066 особенно опасна тем, что злоумышленнику может не понадобиться учетная запись или привилегированный доступ», — пояснил Энсар Секер [<https://www.sans.org/profiles/ensar-seker>], директор по информационной безопасности в SOCRadar.

Злоумышленники могут использовать небезопасный конвейер, загружая специально созданные файлы, которые обманом заставляют Active Storage предоставлять им доступ к файлам, к которым разрешен доступ процессу Rails, даже к конфиденциальным файлам в средах обработки приложений.

На практике это может привести к раскрытию переменных среды, секретов приложения Rails, учетных данных для доступа к базе данных, ключей доступа к облаку, токенов API и учетных данных для подключенных сервисов, пояснил Секер.

Злоумышленники также могут получить доступ к `secret_key_base`, которая подписывает и шифрует файлы cookie, учетные данные [<https://www.csoonline.com/article/570795/how-to-hack-2fa.html>] и данные сеанса. Если `secret_key_base` скомпрометирована, злоумышленники фактически получают доступ к ключу от приложения.

«Непосредственная уязвимость связана с произвольным чтением файлов, но кража секретных данных, таких как `secret_key_base` в Rails, может привести к гораздо более масштабному компрометату», — сказал Секер.

Depending on the application, attackers could potentially forge trusted application data or sessions, access databases and cloud services, move laterally into connected systems, or achieve RCE.

That escalation path is what makes the vulnerability critical, Seker said. “A seemingly routine image upload feature, such as a profile picture, avatar or thumbnail generator, could become an entry point into the application’s underlying infrastructure.”

How to identify if you’re vulnerable

Applications are impacted when they are configured to use *libvips* for Active Storage image processing (the default behavior since Rails 7.0) and accept image uploads from untrusted or unauthenticated users. Enterprises should audit every internal and third-party app to determine whether they are configured this way, Seker advised, and patch Rails and Active Storage immediately. They should also examine every feature accepting images, including avatars, support attachments, product images, and administrative upload functions.

Upgrading Rails alone is not sufficient when an older *libvips* installation remains underneath it; *libvips* must be version 8.13 or later, he said.

Forensic guidance and tooling from the Rails project can help enterprises determine whether apps are vulnerable or files are exploitable, Seker noted. It's also important to review app, proxy, object-storage, and image-processing logs for suspicious uploads or unusual requests.

Additionally, admins should rotate `secret_key_base` and every other credential available in Rails, invalidate active sessions, and investigate downstream systems for potentially exposed credentials.

"Security teams should treat this as a potential secret-exposure incident, not merely a patch-management exercise," Seker said.

Don't trust image processing pipelines

Complex image libraries support many formats and rely on numerous parsers and third-party components, creating a broad attack surface, Seker noted. Therefore, the libraries "should be treated as untrusted code execution territory."

Image processing should be isolated in dedicated sandboxes, containers, or restricted to workers with minimal filesystem access, he advised. There should be no unnecessary network connectivity or access to an app's files or secrets. Strict allowlists should be applied, file content human-validated, and uploads scanned before processing and stored outside app directories.

Additional controls should include short-lived and narrowly scoped credentials, outbound network restrictions, dependency and software composition monitoring, and automated tests that confirm that dangerous codecs or operations are disabled post-upgrade, Seker said.

"The broader lesson is that organizations cannot assess exposure solely by asking whether they 'use Rails,'" he noted, pointing out that two applications running the same Rails version may have very different exposure depending on their image processor, upload paths, and operating system packages. This makes visibility into runtime configuration, libraries, and app functionality critical.

This incident also demonstrates the importance of secret rotation in vulnerability response, he added. "When a vulnerability enables arbitrary file access, installing the patch closes the entry point but does not revoke credentials that may already have been copied."

Don't just assume you're safe

This vulnerability illustrates a perfect use case for a software bill of materials (SBOM), which can speed up discovery of vulnerable software and triage it, Beauceron’s Shipley noted. And enterprises could also adopt intelligent web application firewall monitoring and intervention in addition to isolating systems and patching.

“The words you never want to hear in any critical vulnerability are ‘arbitrary code execution’ and ‘remote code execution,’” he said. “Either of those can mean bad news.”

What’s also interesting here is that the disclosure process was hijacked, he pointed out. Rails published **technical details about the flaw and forensic tools** [<https://discuss.rubyonrails.org/t/cve-2026-66066-attack-details-and-tools-to-perform-a-forensic-investigation/91441>] to assess application vulnerability to it and to look for evidence of data exfiltration nearly a month before it planned to, because several researchers had reverse-engineered the attack and published proof of concept code.

The fact that proofs of concept are now available “materially increases the likelihood of opportunistic scanning and exploitation attempts,” Seker noted.

Therefore, he said, “even organizations that see no obvious evidence of compromise should not assume that patching alone removes the risk created by previously exposed secrets.”

Vulnerabilities[<https://www.csoonline.com/vulnerabilities/>]

Security[<https://www.csoonline.com/security/>]

Web Development[<https://www.csoonline.com/web-development/>]

Development Approaches[<https://www.csoonline.com/development-approaches/>]

Software Development[<https://www.csoonline.com/software-development/>]

About



Policies



More



Our Network





© 2026 FoundryCo, Inc. All Rights Reserved.